

1. Comportamiento del LLM y Conceptos Base

- **Naturaleza del LLM:** Están entrenados inicialmente para **completar el siguiente token/palabra** más probable. Para uso conversacional, usamos modelos tipo Chat (`chat.completions`).
- **Temperatura:** Controla la aleatoriedad. Si es **0**, la respuesta es **determinista** (siempre elige lo más probable). Más cerca a 1 = más creativo.
- **Alucinaciones:** Fenómeno donde el modelo genera **información incorrecta**. Las dos reglas de oro son: 1) *Todos los LLM alucinan* y 2) *Tú eres el responsable de los resultados*.

2. LangChain y LCEL (El núcleo)

- **Runnable:** Es la unidad básica de trabajo en LangChain (ej. LLMs, Templates, Parsers).
- **Métodos principales de un Runnable:**
 - `invoke()`: Ejecuta una vez.
 - `batch()`: Recibe una **lista** y ejecuta varias preguntas en paralelo (ahorrando tiempo).
 - `stream()`: Muestra la respuesta a medida que se va generando en tiempo real.
- **LCEL (LangChain Expression Language):** Permite componer cadenas usando el operador `|` (pipe). El orden clásico es: `chain = template | llm | parser`.
- **Conversiones:**
 - `RunnableLambda`: Convierte una **función pura de Python** en un Runnable (ej. para limpiar un texto antes de enviarlo).
 - `RunnableParallel`: Ejecuta tareas al mismo tiempo (ej. poner mayúsculas y contar palabras a la vez). Recibe un **diccionario** como parámetro.

3. Prompts, Roles y Técnicas

- **Roles de Mensajes:**
 - `HumanMessage`: Lo que escribe el usuario.
 - `AIMessage`: La respuesta del modelo.
 - `SystemMessage`: Establece el rol, contexto o comportamiento general del modelo.
- **Memoria (Chatbots):** Para que el LLM recuerde la charla, se le envía el historial alternando mensajes de usuario e IA. En el `PromptTemplate`, esto se define usando un **placeholder**.
- **Técnicas de Prompting:**
 - **Few-Shot:** Instruir implícitamente dando **ejemplos**.
 - **Chain-of-Thought (CoT):** Brindar ejemplos de **razonamiento paso a paso**. Crucial para evitar que el LLM "salte a conclusiones" matemáticas erróneas.
 - **Zero-Shot CoT:** Lograr que razone paso a paso *sin* darle ejemplos, simplemente agregando la frase **"Pensemos paso a paso"**.

4. Salidas Estructuradas (Pydantic y JSON)

- **Objetivo:** Para que otros sistemas entiendan al LLM, necesitamos estructuras estrictas como un **objeto JSON**.
- **Pydantic:** Es la biblioteca de validación de datos más usada en Python. Heredamos de `BaseModel`.
- **El secreto del éxito:** El LLM lee literalmente el **docstring** de la clase y el parámetro `description` de cada `Field(...)`. Si están bien redactados, el LLM sabrá exactamente qué dato extraer.
- **JsonOutputParser:** Se vincula a la clase Pydantic (`pydantic_object=TuClase`). Su método `.get_format_instructions()` genera las reglas estrictas de formato que se inyectan en el prompt.

5. Herramientas (Tools) y Agentes (LangGraph)

- **Herramientas:** Funciones de Python decoradas con `@tool`. Ganan 3 propiedades: `.name`, `.description` y `.args`.
- **Conexión:** Se vinculan al modelo mediante `.bind_tools()`.
- **tool_calls:** OJO AQUÍ. El LLM **no** ejecuta la herramienta. Solo sugiere usarla devolviendo una propiedad llamada `tool_calls` con el nombre y los argumentos.
- **Agentes:** Sistemas donde el LLM actúa como **motor de razonamiento** para decidir qué herramientas usar y qué acciones tomar.
- **LangGraph y ReAct:**
 - ReAct significa **Reason and Act** (Razona y Actúa). Es la arquitectura que automatiza este ciclo.
 - Se implementa fácilmente con `create_react_agent`.
 - Todo el recorrido del agente se guarda en la propiedad **messages** del estado. Cuando la herramienta de Python se ejecuta con éxito, su resultado se inyecta de vuelta al modelo como un **ToolMessage**.